<epam>

MCP & Skills

Module 3

FEBRUARY 2026

MCP - Model Context Protocol

- **Module Focus:** Connecting AI assistants to external tools and data with MCP
- **Key Question:** How do we extend AI capabilities beyond chat?
- **Practical Outcome:** Working sync workflows for Jira, Confluence, and development tools

What You'll Learn in This Module

1. Explain what MCP is and why it matters
2. Configure MCP servers in GitHub Copilot
3. Use Context7 for up-to-date library documentation
4. Use Playwright for browser automation
5. Sync local markdown specs to Jira and Confluence
6. Create reusable sync prompts with out-of-date detection

MCP: Model Context Protocol — Key Facts

Origin & Design

- **Created by:** Anthropic (Soria Parra & Spahr-Summers)
- **Timeline:** Started July 2024, open-sourced Nov 2024
- **Pain Point:** Copying between Claude Desktop and IDE
- **Design:** Influenced by LSP, uses JSON-RPC 2.0

Linux Foundation (Dec 2025)

- Anthropic donated MCP to the **Agentic AI Foundation** under Linux Foundation
- Neutral governance — equal participation regardless of company size
- 37,000 GitHub stars in under 8 months

The USB Metaphor

- **Before USB:** Every device needed a different port
- **After USB:** One standard port, any device
- **Before MCP:** Custom integrations per AI tool
- **After MCP:** One vendor-neutral protocol, any tool

Why Linux Foundation matters (Dec 2025):

- Long-term stability and reduced adoption risk
- Compatibility guarantees as ecosystem grows
- Suitable for regulated industries

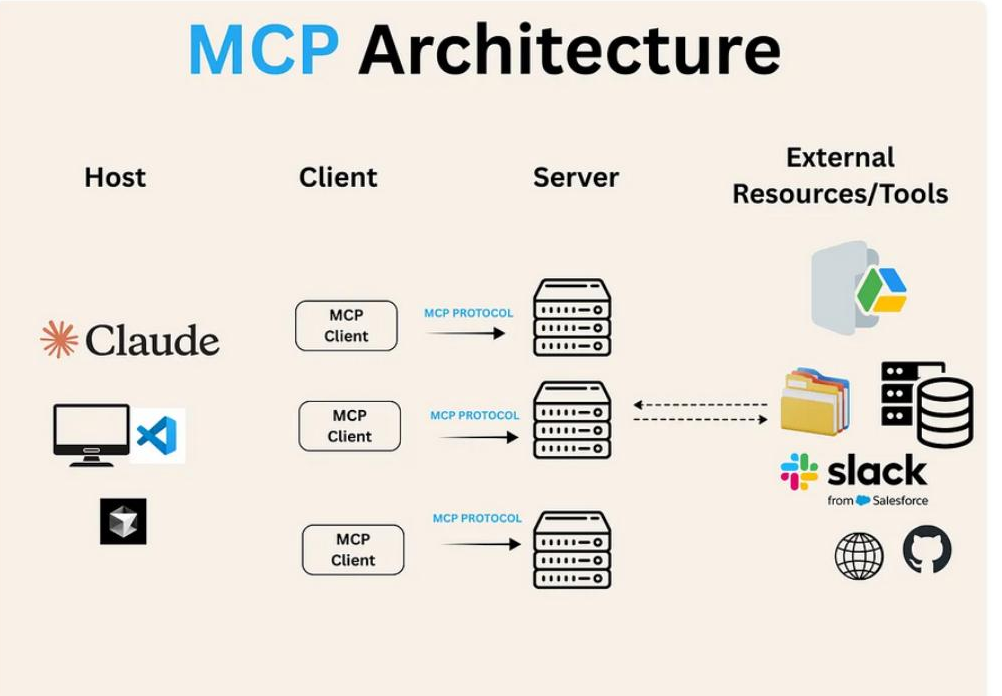
How MCP Works

Client-Server Model:

- **Hosts:** Claude, VS Code, Cursor, your code
- **MCP Clients:** Connect to servers via protocol
- **MCP Servers:** Expose capabilities to clients

What servers expose:

- **Tools:** Actions AI can perform (create issue, take screenshot)
- **Resources:** Data AI can read (docs, DB schemas)
- **Prompts:** Pre-defined templates for common operations



Industry Adoption

Major Platform Support

- **Anthropic:** Claude Desktop, Claude Code — native MCP
- **OpenAI:** ChatGPT, Agents SDK, Responses API (March 2025)
- **Google DeepMind:** Gemini MCP support (April 2025)
- **Microsoft:** Windows, Copilot, VS Code, Azure Foundry (Aug 2025)
- **Other:** Cursor, Zed, Sourcegraph, Replit, and 300+ clients

Growth Numbers (as of 2026)

- **97M+** monthly SDK downloads (Python + TypeScript)
- **10,000+** active MCP servers
- **300+** MCP clients
- **37,000** GitHub stars in under 8 months

Governance: Agentic AI Foundation under Linux Foundation (Dec 2025), co-founded by Anthropic, OpenAI, and Block

Key takeaway: MCP is no longer emerging — it is the industry standard for AI tool integration. Skills you learn today work across all major platforms.

MCP Use Cases — From Dev Tools to Enterprise

AI-Assisted Coding

- Copilot, Cursor, Replit, Sourcegraph, Codeium — all MCP-enabled
- Real-time project context: docs, APIs, databases
- 40-60% faster agent deployment (enterprise reports)

Agentic AI & Multi-Agent Systems

- Agent frameworks (LangChain, AutoGen, CrewAI) use MCP as standard tool interface
- Multi-agent collaboration: one agent diagnoses, another remediates, a third validates
- Gartner: by 2028, 33% of enterprise apps will embed agentic AI

Enterprise & Production

- **Retail:** AI checks inventory, processes refunds, updates loyalty — one conversation
- **Telecom:** AI diagnoses outage, schedules technician, notifies customer
- **DevOps:** AI monitors OpenShift, detects anomalies, posts to Slack via MCP
- **Early adopters:** Block, Apollo, Zed, Replit, Sourcegraph

The MCP Tools We'll Use

Tool	Purpose	Key Benefit
Context7	Up-to-date library docs	No more hallucinated deprecated APIs
Playwright	Browser automation	AI-driven screenshots, form fills, E2E tests
Jira	Issue tracking sync	Local MD specs push to Jira issues
Confluence	Documentation sync	Local MD specs push to Confluence pages
Linear	Alternative PM (optional)	Lightweight Jira alternative

Context7 example: "Use context7 to get React 19 hooks documentation" — fetches current, version-specific docs from the actual source

Playwright example: "Take a screenshot of the login page" or "Fill out the form with test data" — AI drives the browser conversationally

Reusable Prompts & Sync Patterns

Prompt Files — Reusable Automation

Tool	Location
GitHub Copilot	`.github/prompts/*.prompt.md`
Claude Code	`.claude/commands/`
Cursor	`.cursor/commands/`

Parameters use `${input:variableName}` syntax. Invoke with `/` in chat.

Out-of-Date Detection

Four sync states:

- **IN SYNC:** No changes since last sync
- **LOCAL CHANGED:** Local MD edited, remote stale
- **REMOTE CHANGED:** Remote content changed
- **CONFLICT:** Both changed — needs human decision

Key insight: Compare actual content, not remote timestamps (those change for assignee/status/comment updates too).

MCP vs Skills: What's the Difference?

MCP — The Capability Layer

- A **protocol** that connects AI to external tools and data
- Gives AI the **ability** to act: read Jira, write Confluence, take screenshots
- Like **USB ports** on your computer — they enable connections
- Without MCP, AI is isolated in a chat window

Analogy: MCP = the **roads and bridges** of a city. They make travel possible, but don't tell you where to go.

Skills — The Knowledge Layer

- **Rich knowledge packages** that bundle workflows, reference docs, schemas, and scripts
- Tell AI **what to do** with those capabilities — with full domain expertise
- Like **expert guidebooks** — they contain everything needed for a domain
- Without skills, AI has tools but no domain knowledge

Analogy: Skills = the **navigation system with maps, traffic data, and local knowledge**. They use roads (MCP) to get you where you need to go — expertly.

	MCP	Skills
What	Protocol (connections)	Knowledge packages (workflows + reference docs)
Provides	Ability to act	Domain expertise + instructions on how to act
Without it	AI can't reach external tools	AI can reach tools but lacks domain knowledge
Created by	Server developers	Your team, stored in `.claude/skills/`

When to Use MCP, When to Use Skills

Use MCP when you need AI to:

Scenario	MCP Server	Example
Read/write external systems	Jira, Confluence, Linear	"Create a Jira issue"
Automate browsers	Playwright	"Take a screenshot of the login page"
Access live documentation	Context7	"Get React 19 hooks docs"
Query databases	PostgreSQL, SQLite MCP	"Show me the users table schema"
Interact with APIs	Custom MCP servers	"Check deployment status"

Use Skills when you need AI to have domain expertise:

Scenario	Skill Package	Why Not Just MCP?
Create professional documents	`document-skills/docx/`	MCP can write files; skill knows OOXML schemas & formatting
Build presentations	`document-skills/slidev/`	MCP can't know slide layout rules, theme constraints
Follow complex workflows	`document-skills/xlsx/`	Skill bundles decision trees, scripts, and reference docs
Apply format standards	`document-skills/pdf/`	Skill packages conversion logic + quality checks
Encode team conventions	Custom skill packages	MCP can't know your team's processes

umb: MCP = raw capability. Skill = domain expertise package using that capability.

What Are Agent Skills?

Definition: Agent skills are **versioned knowledge packages** stored in `.claude/skills/` that bundle workflows, reference documentation, schemas, and scripts into self-contained domain expertise for AI coding assistants.

Key Characteristics:

Feature	Description
Self-contained	Everything AI needs for a domain in one directory
Rich	SKILL.md + reference docs + schemas + scripts
Versioned	Live in git, reviewed in PRs
Shareable	Entire team gets same domain knowledge
Composable	Skills can use MCP tools and other resources

Think of skills as:

- **Expert knowledge bases** — domain expertise AI can access on demand
- **Apprenticeship packages** — like giving a junior dev a complete reference library
- **Capability bundles** — workflows + documentation + tooling in one place

Without skills:

- AI writes generic code without domain knowledge
- Each developer explains the same constraints repeatedly
- Output quality varies wildly across the team
- Complex formats (OOXML, PDF) require manual fixes

With skills:

- AI has full domain expertise from day one
- Reference schemas and scripts are always available
- Consistent, professional output every time
- Complex formats handled correctly by design

Skill Architecture & Anatomy

Where Skills Live:

```
.claude/skills/
└── <category>/
    └── <skill-name>/
        ├── SKILL.md          ← Entry point
        ├── reference-doc.md  ← Domain docs
        ├── schemas/          ← Format specs
        ├── scripts/          ← Helper tools
        └── LICENSE.txt       ← Terms
```

Real example from this project:

```
.claude/skills/document-skills/
├── docx/
│   ├── SKILL.md
│   ├── docx-js.md
│   ├── ooxml/schemas/...
│   └── LICENSE.txt
├── slidev/
│   └── SKILL.md
├── pdf/
│   └── SKILL.md
└── xlsx/
    └── SKILL.md
```

SKILL.md Anatomy (YAML frontmatter):

```yaml
---
name: docx
description: "Comprehensive document
  creation, editing, and analysis
  with support for tracked changes,
  comments, formatting preservation"
license: Proprietary
---
```

SKILL.md Body — Decision Tree:

```
# DOCX creation, editing, analysis

## Workflow Decision Tree

### Reading/Analyzing Content
Use "Text extraction" section

### Creating New Document
Use "Creating new Word document"

### Editing Existing Document
- Your own doc → Basic OOXML editing
- Someone else's doc → Redlining workflow
- Legal/business docs → Redlining (required)
```

Skills are auto-discovered — AI loads the relevant `SKILL.md` when it

Skills vs Commands: The Evolution

Commands (`.claude/commands/`)

Aspect	Commands
Structure	Single `.md` file
Invocation	`/command-name` (explicit)
Content	Step-by-step instructions
Context	Only what's in the prompt
Best for	Simple, repeatable tasks

```
.claude/commands/
├── review-mr.md
├── course-s0-split-slides.md
└── course-s1-generate-html.md
```

Example — a command is a recipe card:
"Step 1: Read file. Step 2: Check Jira. Step 3: Report."
Simple, linear, explicit invocation.

Skills (`.claude/skills/`)

Aspect	Skills
Structure	Directory with SKILL.md + resources
Invocation	Auto-discovered by AI
Content	Decision trees + reference docs + schemas
Context	Full domain knowledge package
Best for	Complex domains requiring expertise

```
.claude/skills/document-skills/docx/
├── SKILL.md
├── docx-js.md
├── ooxml/schemas/  (30+ XSD files)
└── LICENSE.txt
```

Example — a skill is an expert's complete reference library:
"Here's everything about OOXML: schemas, scripts, workflows, edge cases."
Rich, branching, auto-loaded when relevant.

When to use which: Commands for simple automation (`/review-mr`). Skills for domains requiring deep knowledge (document generation, complex formats).

Building Effective Skills

SKILL.md Best Practices:

- **Clear frontmatter:** `name`, `description`, `license` — AI uses these to match tasks
- **Decision tree, not linear steps:** Branch based on scenario
- **Reference bundled docs:** Point to schemas, scripts in the same directory
- **Handle edge cases:** "If format invalid, use redlining workflow"
- **Keep SKILL.md focused:** Workflow logic only — put details in reference docs

The Skill Quality Test:

1. Does AI auto-discover it for the right tasks?
2. Are reference materials sufficient for correct output?
3. Does the decision tree cover all common scenarios?
4. Can a new team member understand the skill structure?

Building a Skill — Step by Step:

Step	Action
1. **Identify**	Find a domain where AI needs expertise
2. **Collect**	Gather reference docs, schemas, scripts
3. **Structure**	Create `SKILL.md` with decision tree
4. **Bundle**	Add reference materials to skill directory
5. **Test**	Verify AI discovers and uses it correctly
6. **Iterate**	Refine based on output quality

Skill Evolution Pattern:

1. **Ad-hoc:** Explain domain constraints in chat each time
2. **Command:** Extract repeatable steps to `.claude/commands/`
3. **Skill:** Bundle domain knowledge into `.claude/skills/`
4. **Mature:** Add schemas, scripts, reference docs over time
5. **Share:** Team reviews skill packages in PRs

Lab Structure & Summary

Core Labs (95 min):

- Lab 1: Context7 Setup (15 min)
- Lab 2: Playwright Setup (15 min)
- Lab 3: Jira Account & MCP (20 min)
- Lab 4: Confluence MCP (15 min)
- Lab 5: Reusable Sync Prompts (30 min)

Optional: Lab 6 — Linear Alternative PM (15 min)
Labs build on each other — complete 1-5 in order.

What You Learned:

- MCP protocol: "USB for AI" connecting AI to external tools
- Five practical MCP tools for developer productivity
- Sync workflow: Local MD to Jira/Confluence with frontmatter tracking
- Reusable prompt patterns that transfer across tools

What You Built:

- Working MCP configuration in GitHub Copilot
- Sync prompts for Jira and Confluence
- Out-of-date detection prompt

Next Module: Testing Guidelines Creation